

ADVANCED LEVEL – 12 sets

Chapter Framework: Report Card Printer

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that takes a student's name and their scores for three science subjects, then calculates the total and average, and returns a formatted summary string.

- Calculation Steps:
- Calculate the total: `total = physics + chemistry + biology`
 - Calculate the average: `average = total / 3`
 - Return a formatted f-string in exactly this format:
`f'Student: {name} | Total: {total} | Average: {average:.1f}'`

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named <code>calculate_score_summary</code> .
2. The function should accept four parameters: <code>name</code> , <code>physics</code> , <code>chemistry</code> , and <code>biology</code> .
3. The function should calculate the total of the three subject scores.
4. The function should calculate the average by dividing the total by 3.
5. The function should return a formatted f-string with <code>name</code> , <code>total</code> , and <code>average</code> .
6. The average must be displayed to 1 decimal place using <code>:.1f</code> in the f-string.

Tests (8 marks)
1. You should have a function named <code>calculate_score_summary</code> .
2. <code>calculate_score_summary("Alice", 80, 90, 70)</code> → should return <code>"Student: Alice Total: 240 Average: 80.0"</code> .
3. <code>calculate_score_summary("Bob", 60, 75, 90)</code> → should return <code>"Student: Bob Total: 225 Average: 75.0"</code> .
4. <code>calculate_score_summary("Sara", 55, 65, 75)</code> → should return <code>"Student: Sara Total: 195 Average: 65.0"</code> .
5. <code>calculate_score_summary("Tom", 90, 85, 95)</code> → should return <code>"Student: Tom Total: 270 Average: 90.0"</code> .
6. <code>calculate_score_summary("Kim", 70, 70, 70)</code> → should return <code>"Student: Kim Total: 210 Average: 70.0"</code> .

Average: 70.0".
7. calculate_score_summary("Lee", 50, 60, 70) → should return "Student: Lee Total: 180 Average: 60.0".
8. calculate_score_summary("Max", 100, 90, 80) → should return "Student: Max Total: 270 Average: 90.0".

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① A Python file named: [YourClass][YourNumber].py (e.g. S3D05.py)
- ② A screenshot showing your complete code and output in the *Thonny IDE*.

Sample Output

```
Student: Alice | Total: 240 | Average: 80.0
Student: Bob | Total: 225 | Average: 75.0
Student: Sara | Total: 195 | Average: 65.0
Student: Tom | Total: 270 | Average: 90.0
Student: Kim | Total: 210 | Average: 70.0
Student: Lee | Total: 180 | Average: 60.0
Student: Max | Total: 270 | Average: 90.0
```

— — — End of Paper — — —

Chapter Framework: *Employee Profile Generator*

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that creates a visitor pass by building a full name, generating a unique pass code from the visitor's name and year, and returning a formatted pass string.

Pass Building Steps:

- ▶ Build the full name: `full_name = first_name + " " + last_name`
- ▶ Build the pass code using slicing and `str()`:
`badge = first_name[:3].upper() + last_name[:3].upper()`
`year_tag = str(visit_year)[-2:]`
`pass_code = badge + year_tag`
- ▶ Examples: "John" → "JOH" | "Smith" → "SMI" | 2024 → "24"
- ▶ Return the formatted f-string:
`f"Visitor: {full_name} | Pass: {pass_code} | Purpose: {purpose}"`

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named <code>create_visitor_pass</code> .
2. The function should accept four parameters: <code>first_name</code> , <code>last_name</code> , <code>visit_year</code> , and <code>purpose</code> .
3. The function should build <code>full_name</code> by concatenating <code>first_name</code> and <code>last_name</code> .
4. The function should take the first 3 characters of <code>first_name</code> (uppercase) for the badge.
5. The function should take the first 3 characters of <code>last_name</code> (uppercase) for the badge.
6. The function should take the last 2 digits of <code>visit_year</code> using <code>str()</code> and slicing.
7. The function should return a formatted f-string showing <code>full_name</code> , <code>pass_code</code> , and <code>purpose</code> .

Tests (8 marks)
1. You should have a function named <code>create_visitor_pass</code> .
2. <code>create_visitor_pass("John", "Smith", 2024, "Business")</code> → should return <code>"Visitor: John Smith Pass: JOHSMI24 Purpose: Business"</code> .
3. <code>create_visitor_pass("Alice", "Wong", 2023, "Tourism")</code> → should return <code>"Visitor: Alice Wong Pass: ALIWON23 Purpose: Tourism"</code> .

4. create_visitor_pass("Bob", "Lee", 2022, "Study") → should return "Visitor: Bob Lee Pass: BOBLEE22 Purpose: Study".
5. create_visitor_pass("Sara", "Chan", 2021, "Research") → should return "Visitor: Sara Chan Pass: SARCHA21 Purpose: Research".
6. create_visitor_pass("Tom", "Park", 2020, "Business") → should return "Visitor: Tom Park Pass: TOMPAR20 Purpose: Business".
7. create_visitor_pass("Eva", "Kim", 2019, "Tourism") → should return "Visitor: Eva Kim Pass: EVAKIM19 Purpose: Tourism".
8. create_visitor_pass("Mike", "Tan", 2025, "Study") → should return "Visitor: Mike Tan Pass: MIKTAN25 Purpose: Study".

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① A Python file named: [YourClass][YourNumber].py (e.g. S3D05.py)
- ② A screenshot showing your complete code and output in the *Thonny IDE*.

Sample Output

```
Visitor: John Smith | Pass: JOHSMI24 | Purpose: Business
Visitor: Alice Wong | Pass: ALIWON23 | Purpose: Tourism
Visitor: Bob Lee | Pass: BOBLEE22 | Purpose: Study
Visitor: Sara Chan | Pass: SARCHA21 | Purpose: Research
Visitor: Tom Park | Pass: TOMPAR20 | Purpose: Business
Visitor: Eva Kim | Pass: EVAKIM19 | Purpose: Tourism
Visitor: Mike Tan | Pass: MIKTAN25 | Purpose: Study
```

— — — End of Paper — — —

Chapter Framework: *Bill Splitter*

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that calculates how much each guest must pay for a hotel stay, given the costs for the room, food, and spa, a tip percentage, and the number of guests sharing the bill.

Calculation Steps:

- ▶ Use a running total starting at 0.
- ▶ Add each cost one by one using +=:
 running_total += room_cost
 running_total += food_cost
 running_total += spa_cost
- ▶ Calculate the tip: tip = running_total * (tip_percent / 100)
- ▶ Add the tip: running_total += tip
- ▶ Divide by guests and return rounded to 2 d.p.: round(running_total / num_guests, 2)

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named split_hotel_bill.
2. The function should accept five parameters: room_cost, food_cost, spa_cost, num_guests, and tip_percent.
3. The function should use a running_total variable starting at 0.
4. The function should add room_cost, food_cost, and spa_cost to running_total using +=.
5. The function should calculate the tip as a percentage of the running_total.
6. The function should add the tip to running_total, then divide by num_guests.
7. The function should return the per-guest cost rounded to 2 decimal places.

Tests (8 marks)
1. You should have a function named split_hotel_bill.
2. split_hotel_bill(300, 100, 100, 5, 0) → should return 100.0.
3. split_hotel_bill(400, 200, 200, 4, 25) → should return 250.0.
4. split_hotel_bill(600, 300, 100, 10, 10) → should return 110.0.

5. split_hotel_bill(500, 200, 300, 8, 0) → should return 125.0.
6. split_hotel_bill(200, 100, 50, 7, 0) → should return 50.0.
7. split_hotel_bill(800, 400, 200, 6, 20) → should return 280.0.
8. split_hotel_bill(150, 75, 75, 3, 0) → should return 100.0.

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① A Python file named: `[YourClass][YourNumber].py` (e.g. `S3D05.py`)
- ② A screenshot showing your complete code and output in the *Thonny IDE*.

Sample Output

```
100.0
250.0
110.0
125.0
50.0
280.0
100.0
```

— — — **End of Paper** — — —

Chapter Framework: *Movie Ticket Booking Calculator*

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that calculates the price of a cinema ticket based on the customer's age, the day of the week, and whether they hold a membership card.

Ticket Pricing Rules:

- ▶ age must be an integer. If not, return: Invalid age
- ▶ is_member must be a boolean. If not, return: Invalid member status
- ▶ day must be "weekday" or "weekend". If not, return: Invalid day
- ▶ If age is below 0, return: Invalid age
- ▶ Base prices: age < 12 → \$8.00 | age >= 65 → \$10.00 | otherwise → \$15.00
- ▶ A \$2.00 weekday discount applies to adults (age >= 12) when day == "weekday".
- ▶ A \$1.50 member discount applies to all valid tickets when is_member is True.

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named get_cinema_ticket.
2. The function should accept three parameters: age, day, and is_member.
3. If age is not an integer or is below 0, return the string Invalid age.
4. If is_member is not a boolean, return the string Invalid member status.
5. If day is not 'weekday' or 'weekend', return the string Invalid day.
6. The base price depends on age: child (\$8.00), adult (\$15.00), or senior (\$10.00).
7. Adult and senior tickets get a \$2.00 discount on weekdays; members get a \$1.50 discount.

Tests (8 marks)
1. You should have a function named get_cinema_ticket.
2. get_cinema_ticket("ten", "weekday", True) → should return "Invalid age".
3. get_cinema_ticket(25, "tuesday", True) → should return "Invalid day".
4. get_cinema_ticket(25, "weekday", "yes") → should return "Invalid member status".
5. get_cinema_ticket(8, "weekend", False) → should return 8.0.

6. <code>get_cinema_ticket(20, "weekend", False)</code> → should return 15.0.
7. <code>get_cinema_ticket(20, "weekday", True)</code> → should return 11.5.
8. <code>get_cinema_ticket(70, "weekday", False)</code> → should return 8.0.

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① A Python file named: `[YourClass][YourNumber].py` (e.g. `S3D05.py`)
- ② A screenshot showing your complete code and output in the *Thonny IDE*.

Sample Output

```
Invalid age
Invalid day
Invalid member status
8.0
15.0
11.5
8.0
```

— — — **End of Paper** — — —

Chapter Framework: *Travel Weather Planner*

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that gives travel advice based on the temperature, whether it is raining, and the travel distance. Each variable is checked with its own separate conditional chain.

Advisory Rules:

- ▶ temperature must be a number (int or float). If not, return: Invalid temperature
- ▶ is_raining must be a boolean. If not, return: Invalid weather status
- ▶ distance must be a positive number (int or float). If not valid, return: Invalid distance
- ▶ If is_raining is True, return: Bring an umbrella. Check road conditions.
- ▶ If temperature > 35, return: Too hot to travel. Stay hydrated.
- ▶ If temperature < 5, return: Too cold. Check for icy roads.
- ▶ If distance > 100 and temperature >= 20:
 return: Long journey. Pack snacks and water.
- ▶ If distance > 100 and temperature < 20:
 return: Long journey in cool weather. Bring warm clothing.
- ▶ Otherwise, return: Pleasant short trip ahead!

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named advise_trip.
2. The function should accept three parameters: temperature, is_raining, and distance.
3. If temperature is not a number (int or float), return the string Invalid temperature.
4. If is_raining is not a boolean, return the string Invalid weather status.
5. If distance is not a positive number, return the string Invalid distance.
6. If is_raining is True, always return Bring an umbrella. Check road conditions.
7. Temperature and distance checks determine the remaining advice messages.

Tests (8 marks)
1. You should have a function named advise_trip.
2. advise_trip("hot", False, 50) → should return "Invalid temperature".

3. advise_trip(25, "no", 50) → should return "Invalid weather status".
4. advise_trip(25, False, -10) → should return "Invalid distance".
5. advise_trip(25, True, 50) → should return "Bring an umbrella. Check road conditions.".
6. advise_trip(38, False, 50) → should return "Too hot to travel. Stay hydrated.".
7. advise_trip(25, False, 150) → should return "Long journey. Pack snacks and water.".
8. advise_trip(3, False, 50) → should return "Too cold. Check for icy roads.".

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① **A Python file named:** `[YourClass][YourNumber].py` (e.g. **S3D05.py**)
- ② **A screenshot** showing your complete code and output in the *Thonny IDE*.

Sample Output

```
Invalid temperature
Invalid weather status
Invalid distance
Bring an umbrella. Check road conditions.
Too hot to travel. Stay hydrated.
Long journey. Pack snacks and water.
Too cold. Check for icy roads.
```

— — — **End of Paper** — — —

Chapter Framework: *Apply Discount Function*

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that calculates the final price after tax. The function must fully validate both inputs before doing any calculation, following the same pattern as the freeCodeCamp `apply_discount` function.

Validation & Calculation Rules:

- ▶ price must be a number (int or float). If not, return: The price must be a number
- ▶ tax_rate must be a number (int or float). If not, return: The tax rate must be a number
- ▶ price must be greater than 0. If not, return: The price must be greater than 0
- ▶ tax_rate must be 0 or greater. If not, return: The tax rate must not be negative
- ▶ tax = price * (tax_rate / 100)
- ▶ Return `round(price + tax, 2)`

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named <code>apply_tax</code> .
2. The function should accept two parameters: <code>price</code> and <code>tax_rate</code> .
3. If <code>price</code> is not a number (int or float), return the string The price must be a number.
4. If <code>tax_rate</code> is not a number (int or float), return the string The tax rate must be a number.
5. If <code>price</code> is less than or equal to 0, return the string The price must be greater than 0.
6. If <code>tax_rate</code> is negative, return the string The tax rate must not be negative.
7. The function must return the final price rounded to 2 decimal places.

Tests (8 marks)
1. You should have a function named <code>apply_tax</code> .
2. <code>apply_tax("100", 10)</code> → should return "The price must be a number".
3. <code>apply_tax(100, "ten")</code> → should return "The tax rate must be a number".
4. <code>apply_tax(-50, 10)</code> → should return "The price must be greater than 0".
5. <code>apply_tax(100, -5)</code> → should return "The tax rate must not be negative".
6. <code>apply_tax(100, 10)</code> → should return 110.0.

7. <code>apply_tax(250, 8)</code> → should return 270.0.
8. <code>apply_tax(80, 0)</code> → should return 80.0.

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① **A Python file named:** `[YourClass][YourNumber].py` (e.g. **S3D05.py**)
- ② **A screenshot** showing your complete code and output in the *Thonny IDE*.

Sample Output

```
The price must be a number
The tax rate must be a number
The price must be greater than 0
The tax rate must not be negative
110.0
270.0
80.0
```

— — — **End of Paper** — — —

Chapter Framework: *Bill Splitter*

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that calculates an employee's net salary. The function takes their base salary, how many overtime hours they worked, the overtime hourly rate, and the tax rate.

Payslip Calculation Steps:

- ▶ Calculate overtime pay: `overtime_pay = overtime_hours * hourly_rate`
- ▶ Calculate gross salary: `gross_salary = base_salary + overtime_pay`
- ▶ Calculate tax deduction: `tax_deduction = gross_salary * (tax_rate / 100)`
- ▶ Calculate net salary: `net_salary = gross_salary - tax_deduction`
- ▶ Return `net_salary` rounded to 2 decimal places using `round()`.

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named <code>calculate_payslip</code> .
2. The function should accept four parameters: <code>base_salary</code> , <code>overtime_hours</code> , <code>hourly_rate</code> , and <code>tax_rate</code> .
3. The function should calculate overtime pay by multiplying <code>overtime_hours</code> by <code>hourly_rate</code> .
4. The function should calculate gross salary by adding overtime pay to <code>base_salary</code> .
5. The function should calculate tax as a percentage of gross salary.
6. The function should subtract tax from gross salary to get net salary.
7. The function should return <code>net_salary</code> rounded to 2 decimal places.

Tests (8 marks)
1. You should have a function named <code>calculate_payslip</code> .
2. <code>calculate_payslip(3000, 0, 50, 0)</code> → should return <code>3000.0</code> .
3. <code>calculate_payslip(3000, 10, 50, 20)</code> → should return <code>2800.0</code> .
4. <code>calculate_payslip(4000, 5, 60, 15)</code> → should return <code>3655.0</code> .
5. <code>calculate_payslip(5000, 0, 80, 25)</code> → should return <code>3750.0</code> .
6. <code>calculate_payslip(2500, 8, 40, 10)</code> → should return <code>2538.0</code> .

7. calculate_payslip(6000, 20, 100, 30) → should return 5600.0.
8. calculate_payslip(3500, 15, 70, 20) → should return 3640.0.

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① A Python file named: `[YourClass][YourNumber].py` (e.g. `S3D05.py`)
- ② A screenshot showing your complete code and output in the *Thonny IDE*.

Sample Output

```
3000.0
2800.0
3655.0
3750.0
2538.0
5600.0
3640.0
```

— — — **End of Paper** — — —

Chapter Framework: *Apply Discount Function*

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that calculates the total membership fee. The function must validate all inputs, apply the correct promo code discount, and return the final fee rounded to 2 decimal places.

Validation & Pricing Rules:

- ▶ base_fee must be a number (int or float). If not, return: The fee must be a number
- ▶ duration_months must be an integer. If not, return: The duration must be an integer
- ▶ base_fee must be greater than 0. If not, return: The fee must be greater than 0
- ▶ duration_months must be >= 1. If not, return: The duration must be at least 1 month
- ▶ promo_code must be "SAVE10", "SAVE20", or "NONE". If not, return: Invalid promo code
- ▶ total = base_fee * duration_months
- ▶ "SAVE10" → total = total * 0.9
- ▶ "SAVE20" → total = total * 0.8
- ▶ Return round(total, 2)

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named calculate_membership_fee.
2. The function should accept three parameters: base_fee, duration_months, and promo_code.
3. If base_fee is not a number (int or float), return the string The fee must be a number.
4. If duration_months is not an integer, return the string The duration must be an integer.
5. If base_fee is not greater than 0, return The fee must be greater than 0.
6. If duration_months is less than 1, return The duration must be at least 1 month.
7. If promo_code is not SAVE10, SAVE20, or NONE, return the string Invalid promo code.
8. The function must apply the correct discount and return the total rounded to 2 d.p.

Tests (8 marks)
1. You should have a function named calculate_membership_fee.
2. calculate_membership_fee("100", 3, "NONE") → should return "The fee must be a number".
3. calculate_membership_fee(100, "3", "NONE") → should return "The duration must be an integer".

4. calculate_membership_fee(-100, 3, "NONE") → should return "The fee must be greater than 0".
5. calculate_membership_fee(100, 0, "NONE") → should return "The duration must be at least 1 month".
6. calculate_membership_fee(100, 3, "VIP") → should return "Invalid promo code".
7. calculate_membership_fee(100, 6, "SAVE10") → should return 540.0.
8. calculate_membership_fee(200, 3, "SAVE20") → should return 480.0.

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① **A Python file named:** `[YourClass][YourNumber].py` (e.g. **S3D05.py**)
- ② **A screenshot** showing your complete code and output in the *Thonny IDE*.

Sample Output

```
The fee must be a number
The duration must be an integer
The fee must be greater than 0
The duration must be at least 1 month
Invalid promo code
540.0
480.0
```

— — — **End of Paper** — — —

Chapter Framework: Ch.2 Report Card + Ch.3 Employee Profile

⚡ **Combination Challenge** — This task requires skills from Ch.2 (arithmetic, f-string `:.1f`) and Ch.3 (string slicing, `+`). Figure out which chapter each part of the solution comes from!

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that combines string operations and arithmetic to produce a formatted grade report. The function creates initials from the student's name (string slicing), calculates the average (arithmetic), and formats the result with an f-string.

Report Building Steps:

- ▶ Build the full name: `full_name = first_name + " " + last_name`
- ▶ Build initials using string slicing:
`initials = first_name[0] + last_name[0]`
- ▶ Calculate total: `total = score1 + score2 + score3`
- ▶ Calculate average: `average = total / 3`
- ▶ Return the f-string:
`f'{initials} | {full_name} | Total: {total} | Average: {average:.1f}'`

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named <code>format_grade_report</code> .
2. The function should accept five parameters: <code>first_name</code> , <code>last_name</code> , <code>score1</code> , <code>score2</code> , and <code>score3</code> .
3. The function should build a <code>full_name</code> by concatenating <code>first_name</code> and <code>last_name</code> .
4. The function should build initials using the first character of each name (slicing).
5. The function should calculate the total of the three scores.
6. The function should calculate the average by dividing total by 3.
7. The function should return a formatted f-string showing initials, <code>full_name</code> , total, and average (<code>:.1f</code>).

Tests (8 marks)
1. You should have a function named <code>format_grade_report</code> .
2. <code>format_grade_report("Alice", "Brown", 80, 90, 70)</code> → should return <code>"AB Alice Brown Total: 240"</code>

Average: 80.0".
3. format_grade_report("Bob", "Smith", 60, 75, 90) → should return "BS Bob Smith Total: 225 Average: 75.0".
4. format_grade_report("Sara", "Lee", 55, 65, 75) → should return "SL Sara Lee Total: 195 Average: 65.0".
5. format_grade_report("Tom", "Wong", 90, 85, 95) → should return "TW Tom Wong Total: 270 Average: 90.0".
6. format_grade_report("Kim", "Park", 70, 70, 70) → should return "KP Kim Park Total: 210 Average: 70.0".
7. format_grade_report("Eva", "Chan", 50, 60, 70) → should return "EC Eva Chan Total: 180 Average: 60.0".
8. format_grade_report("Mike", "Kim", 100, 90, 80) → should return "MK Mike Kim Total: 270 Average: 90.0".

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① A Python file named: [YourClass][YourNumber].py (e.g. S3D05.py)
- ② A screenshot showing your complete code and output in the *Thonny IDE*.

Sample Output

```
AB | Alice Brown | Total: 240 | Average: 80.0
BS | Bob Smith | Total: 225 | Average: 75.0
SL | Sara Lee | Total: 195 | Average: 65.0
TW | Tom Wong | Total: 270 | Average: 90.0
KP | Kim Park | Total: 210 | Average: 70.0
EC | Eva Chan | Total: 180 | Average: 60.0
MK | Mike Kim | Total: 270 | Average: 90.0
```

— — — End of Paper — — —

Chapter Framework: Ch.3 Employee Profile + Ch.4 Bill Splitter

⚡ **Combination Challenge** — This task requires skills from Ch.3 (string slicing [:3], .upper()) and Ch.4 (*, /, round()). Figure out which chapter each part of the solution comes from!

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that combines string slicing and mathematical operations to generate an order summary. The function extracts a category tag from the product code, calculates the discounted total, and returns a formatted string.

Order Calculation Steps:

- ▶ Extract the category tag: category = product_code[0:3].upper()
- ▶ Calculate subtotal: subtotal = unit_price * quantity
- ▶ Calculate discount: discount = subtotal * (discount_rate / 100)
- ▶ Calculate total: total = round(subtotal - discount, 2)
- ▶ Return the f-string:
f'[{category}] Total: {total}'

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named calculate_order.
2. The function should accept four parameters: product_code, unit_price, quantity, and discount_rate.
3. The function should extract the first 3 characters of product_code and convert to uppercase.
4. The function should calculate the subtotal by multiplying unit_price and quantity.
5. The function should calculate the discount as a percentage of the subtotal.
6. The function should subtract the discount from the subtotal to get the total.
7. The function should return a formatted f-string showing the category tag and total.

Tests (8 marks)
1. You should have a function named calculate_order.
2. calculate_order("electronics001", 50, 4, 10) → should return "[ELE] Total: 180.0".

3. calculate_order("books002", 20, 5, 0) → should return "[BOO] Total: 100.0".
4. calculate_order("clothing003", 80, 3, 25) → should return "[CLO] Total: 180.0".
5. calculate_order("food004", 10, 10, 5) → should return "[FOO] Total: 95.0".
6. calculate_order("sports005", 100, 2, 20) → should return "[SPO] Total: 160.0".
7. calculate_order("toys006", 30, 6, 0) → should return "[TOY] Total: 180.0".
8. calculate_order("medical007", 150, 1, 15) → should return "[MED] Total: 127.5".

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① A Python file named: [YourClass][YourNumber].py (e.g. S3D05.py)
- ② A screenshot showing your complete code and output in the *Thonny IDE*.

Sample Output

```
[ELE] Total: 180.0
[BOO] Total: 100.0
[CLO] Total: 180.0
[FOO] Total: 95.0
[SPO] Total: 160.0
[TOY] Total: 180.0
[MED] Total: 127.5
```

— — — End of Paper — — —

Chapter Framework: Ch.4 Bill Splitter + Ch.5 Conditionals

 **Combination Challenge** — This task requires skills from Ch.4 (cost = units * rate, +=, round()) and Ch.5 (isinstance, if/else for rate). Figure out which chapter each part of the solution comes from!

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that calculates an electricity bill. The rate depends on whether the account is residential or commercial (conditional logic). If usage exceeds 500 units, a surcharge applies (arithmetic). Both inputs must be validated.

Billing Rules:

- units_used must be a number (int or float). If not, return: Invalid units
- is_residential must be a boolean. If not, return: Invalid account type
- units_used must be >= 0. If not, return: Invalid units
- Set the rate: is_residential == True → \$0.12 | False → \$0.22
- Calculate cost: cost = units_used * rate
- Add surcharge: if units_used > 500: cost += 15.0
- Return round(cost, 2)

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named calculate_energy_bill.
2. The function should accept two parameters: units_used and is_residential.
3. If units_used is not a number (int or float), return the string Invalid units.
4. If is_residential is not a boolean, return the string Invalid account type.
5. If units_used is below 0, return the string Invalid units.
6. The rate should be 0.12 for residential accounts and 0.22 for commercial accounts.
7. A surcharge of \$15.00 should be added when units_used exceeds 500.

Tests (8 marks)
1. You should have a function named calculate_energy_bill.

2. calculate_energy_bill("500", True) → should return "Invalid units".
3. calculate_energy_bill(500, "yes") → should return "Invalid account type".
4. calculate_energy_bill(-100, True) → should return "Invalid units".
5. calculate_energy_bill(200, True) → should return 24.0.
6. calculate_energy_bill(200, False) → should return 44.0.
7. calculate_energy_bill(600, True) → should return 87.0.
8. calculate_energy_bill(600, False) → should return 147.0.

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① **A Python file named:** `[YourClass][YourNumber].py` (e.g. `S3D05.py`)
- ② **A screenshot** showing your complete code and output in the *Thonny IDE*.

Sample Output

```
Invalid units
Invalid account type
Invalid units
24.0
44.0
87.0
147.0
```

— — — **End of Paper** — — —

Chapter Framework: Ch.2 Report Card + Ch.5 Conditionals + Ch.6 Functions

 **Combination Challenge** — This task requires skills from Ch.2 (f-string `:.1f`) + Ch.5 (if/elif grade chain) + Ch.6 (isinstance validation). Figure out which chapter each part of the solution comes from!

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that calculates a student's final grade. The function validates all inputs (Ch.6 pattern), adjusts the score with bonus marks and an attendance bonus (arithmetic), determines the letter grade using conditionals (Ch.5), and returns a formatted string using f-string with `:.1f` (Ch.2).

Grade Calculation Rules:

- raw_score must be a number (int or float). If not, return: Invalid score
- bonus_marks must be a number (int or float). If not, return: Invalid bonus
- has_attended_all must be a boolean. If not, return: Invalid attendance status
- raw_score must be between 0 and 100 (inclusive). If not, return: Score out of range
- adjusted = raw_score + bonus_marks
 - If has_attended_all is True: adjusted += 5
- Letter grades: A (≥ 90) | B (≥ 80) | C (≥ 70) | D (≥ 60) | F (below 60)
- Return: f"Score: {adjusted:.1f} | Grade: {grade}"

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named <code>calculate_final_grade</code> .
2. The function should accept three parameters: <code>raw_score</code> , <code>bonus_marks</code> , and <code>has_attended_all</code> .
3. If <code>raw_score</code> is not a number (int or float), return the string Invalid score.
4. If <code>bonus_marks</code> is not a number (int or float), return the string Invalid bonus.
5. If <code>has_attended_all</code> is not a boolean, return the string Invalid attendance status.
6. If <code>raw_score</code> is below 0 or above 100, return the string Score out of range.
7. Attendance bonus of 5 marks applies when <code>has_attended_all</code> is True.
8. Return the formatted f-string with the adjusted score (<code>:.1f</code>) and the letter grade.

1. You should have a function named <code>calculate_final_grade</code> .
2. <code>calculate_final_grade("80", 5, True)</code> → should return "Invalid score".
3. <code>calculate_final_grade(80, "five", True)</code> → should return "Invalid bonus".
4. <code>calculate_final_grade(80, 5, "yes")</code> → should return "Invalid attendance status".
5. <code>calculate_final_grade(110, 5, True)</code> → should return "Score out of range".
6. <code>calculate_final_grade(85, 5, True)</code> → should return "Score: 95.0 Grade: A".
7. <code>calculate_final_grade(75, 0, False)</code> → should return "Score: 75.0 Grade: C".
8. <code>calculate_final_grade(50, 5, False)</code> → should return "Score: 55.0 Grade: F".

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① A Python file named: [YourClass][YourNumber].py (e.g. S3D05.py)
- ② A screenshot showing your complete code and output in the *Thonny IDE*.

Sample Output

```
Invalid score
Invalid bonus
Invalid attendance status
Score out of range
Score: 95.0 | Grade: A
Score: 75.0 | Grade: C
Score: 55.0 | Grade: F
```

— — — End of Paper — — —

ANSWER KEY

Model answers for all 12 sets. Accept any logically equivalent solution.

Set 1 — Build a Science Score Summariser

Chapter: *Ch.2 — Variables & Data Types* | Framework: *Report Card Printer*

Function: `calculate_score_summary(name, physics, chemistry, biology)`

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def calculate_score_summary(name, physics, chemistry, biology):
    total    = physics + chemistry + biology
    average  = total / 3
    return f"Student: {name} | Total: {total} | Average: {average:.1f}"

print(calculate_score_summary("Alice", 80, 90, 70))
print(calculate_score_summary("Bob", 60, 75, 90))
print(calculate_score_summary("Sara", 55, 65, 75))
print(calculate_score_summary("Tom", 90, 85, 95))
print(calculate_score_summary("Kim", 70, 70, 70))
print(calculate_score_summary("Lee", 50, 60, 70))
print(calculate_score_summary("Max", 100, 90, 80))
```

Set 2 — Build a Visitor Pass Generator

Chapter: *Ch.3 — Introduction to Strings* | Framework: *Employee Profile Generator*

Function: `create_visitor_pass(first_name, last_name, visit_year, purpose)`

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def create_visitor_pass(first_name, last_name, visit_year, purpose):
    full_name = first_name + " " + last_name
    badge      = first_name[:3].upper() + last_name[:3].upper()
    year_tag   = str(visit_year)[-2:]
    pass_code  = badge + year_tag
    return f"Visitor: {full_name} | Pass: {pass_code} | Purpose: {purpose}"

print(create_visitor_pass("John", "Smith", 2024, "Business"))
print(create_visitor_pass("Alice", "Wong", 2023, "Tourism"))
print(create_visitor_pass("Bob", "Lee", 2022, "Study"))
print(create_visitor_pass("Sara", "Chan", 2021, "Research"))
print(create_visitor_pass("Tom", "Park", 2020, "Business"))
print(create_visitor_pass("Eva", "Kim", 2019, "Tourism"))
print(create_visitor_pass("Mike", "Tan", 2025, "Study"))
```

Set 3 — Build a Hotel Bill Splitter

Chapter: *Ch.4 — Numbers & Mathematical Operations* | Framework: *Bill Splitter*

Function: `split_hotel_bill(room_cost, food_cost, spa_cost, num_guests, tip_percent)`

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def split_hotel_bill(room_cost, food_cost, spa_cost, num_guests, tip_percent):
    running_total = 0
```

```

    running_total += room_cost
    running_total += food_cost
    running_total += spa_cost
    tip            = running_total * (tip_percent / 100)
    running_total += tip
    return round(running_total / num_guests, 2)

print(split_hotel_bill(300, 100, 100, 5, 0))
print(split_hotel_bill(400, 200, 200, 4, 25))
print(split_hotel_bill(600, 300, 100, 10, 10))
print(split_hotel_bill(500, 200, 300, 8, 0))
print(split_hotel_bill(200, 100, 50, 7, 0))
print(split_hotel_bill(800, 400, 200, 6, 20))
print(split_hotel_bill(150, 75, 75, 3, 0))

```

Set 4 — Build a Cinema Ticket Calculator

Chapter: *Ch.5 — Booleans & Conditionals* | **Framework:** *Movie Ticket Booking Calculator*

Function: `get_cinema_ticket(age, day, is_member)`

```

print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def get_cinema_ticket(age, day, is_member):
    if not isinstance(age, int):
        return "Invalid age"
    if not isinstance(is_member, bool):
        return "Invalid member status"
    if day not in ("weekday", "weekend"):
        return "Invalid day"
    if age < 0:
        return "Invalid age"
    if age < 12:
        price = 8.0
    elif age >= 65:
        price = 10.0
    else:
        price = 15.0
    if day == "weekday" and age >= 12:
        price -= 2.0
    if is_member:
        price -= 1.5
    return price

print(get_cinema_ticket("ten", "weekday", True))
print(get_cinema_ticket(25, "tuesday", True))
print(get_cinema_ticket(25, "weekday", "yes"))
print(get_cinema_ticket(8, "weekend", False))
print(get_cinema_ticket(20, "weekend", False))
print(get_cinema_ticket(20, "weekday", True))
print(get_cinema_ticket(70, "weekday", False))

```

Set 5 — Build a Trip Advisor

Chapter: *Ch.5 Lab — Multiple Conditional Chains* | **Framework:** *Travel Weather Planner*

Function: `advise_trip(temperature, is_raining, distance)`

```

print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

```

```
def advise_trip(temperature, is_raining, distance):
    if not isinstance(temperature, (int, float)):
        return "Invalid temperature"
    if not isinstance(is_raining, bool):
        return "Invalid weather status"
    if not isinstance(distance, (int, float)) or distance <= 0:
        return "Invalid distance"
    if is_raining:
        return "Bring an umbrella. Check road conditions."
    if temperature > 35:
        return "Too hot to travel. Stay hydrated."
    if temperature < 5:
        return "Too cold. Check for icy roads."
    if distance > 100:
        if temperature >= 20:
            return "Long journey. Pack snacks and water."
        return "Long journey in cool weather. Bring warm clothing."
    return "Pleasant short trip ahead!"

print(advise_trip("hot", False, 50))
print(advise_trip(25, "no", 50))
print(advise_trip(25, False, -10))
print(advise_trip(25, True, 50))
print(advise_trip(38, False, 50))
print(advise_trip(25, False, 150))
print(advise_trip(3, False, 50))
```

Set 6 — Build a Tax Calculator

Chapter: Ch.6 — Understanding Functions & Scope | Framework: Apply Discount Function

Function: apply_tax(price, tax_rate)

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def apply_tax(price, tax_rate):
    if not isinstance(price, (int, float)):
        return "The price must be a number"
    if not isinstance(tax_rate, (int, float)):
        return "The tax rate must be a number"
    if price <= 0:
        return "The price must be greater than 0"
    if tax_rate < 0:
        return "The tax rate must not be negative"
    tax = price * (tax_rate / 100)
    return round(price + tax, 2)

print(apply_tax("100", 10))
print(apply_tax(100, "ten"))
print(apply_tax(-50, 10))
print(apply_tax(100, -5))
print(apply_tax(100, 10))
print(apply_tax(250, 8))
print(apply_tax(80, 0))
```

Set 7 — Build a Payslip Calculator

Chapter: Ch.4 — Numbers & Mathematical Operations (variant) | Framework: Bill Splitter

Function: calculate_payslip(base_salary, overtime_hours, hourly_rate, tax_rate)

```
print("Name: [Your Name]")
```

```
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def calculate_payslip(base_salary, overtime_hours, hourly_rate, tax_rate):
    overtime_pay = overtime_hours * hourly_rate
    gross_salary = base_salary + overtime_pay
    tax_deduction = gross_salary * (tax_rate / 100)
    net_salary = gross_salary - tax_deduction
    return round(net_salary, 2)

print(calculate_payslip(3000, 0, 50, 0))
print(calculate_payslip(3000, 10, 50, 20))
print(calculate_payslip(4000, 5, 60, 15))
print(calculate_payslip(5000, 0, 80, 25))
print(calculate_payslip(2500, 8, 40, 10))
print(calculate_payslip(6000, 20, 100, 30))
print(calculate_payslip(3500, 15, 70, 20))
```

Set 8 — Build a Membership Fee Calculator

Chapter: *Ch.6 — Understanding Functions & Scope (variant)* | Framework: *Apply Discount Function*
Function: `calculate_membership_fee(base_fee, duration_months, promo_code)`

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def calculate_membership_fee(base_fee, duration_months, promo_code):
    if not isinstance(base_fee, (int, float)):
        return "The fee must be a number"
    if not isinstance(duration_months, int):
        return "The duration must be an integer"
    if base_fee <= 0:
        return "The fee must be greater than 0"
    if duration_months < 1:
        return "The duration must be at least 1 month"
    if promo_code not in ("SAVE10", "SAVE20", "NONE"):
        return "Invalid promo code"
    total = base_fee * duration_months
    if promo_code == "SAVE10":
        total = total * 0.9
    elif promo_code == "SAVE20":
        total = total * 0.8
    return round(total, 2)

print(calculate_membership_fee("100", 3, "NONE"))
print(calculate_membership_fee(100, "3", "NONE"))
print(calculate_membership_fee(-100, 3, "NONE"))
print(calculate_membership_fee(100, 0, "NONE"))
print(calculate_membership_fee(100, 3, "VIP"))
print(calculate_membership_fee(100, 6, "SAVE10"))
print(calculate_membership_fee(200, 3, "SAVE20"))
```

Set 9 ⚡ COMBINATION — Build a Grade Report Formatter

Chapter: *Ch.2 + Ch.3 — Combination Challenge* | Framework: *Ch.2 Report Card + Ch.3 Employee Profile*
Function: `format_grade_report(first_name, last_name, score1, score2, score3)`

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```



```
def format_grade_report(first_name, last_name, score1, score2, score3):
    full_name = first_name + " " + last_name
    initials = first_name[0] + last_name[0]
    total = score1 + score2 + score3
    average = total / 3
    return f"{initials} | {full_name} | Total: {total} | Average: {average:.1f}"

print(format_grade_report("Alice", "Brown", 80, 90, 70))
print(format_grade_report("Bob", "Smith", 60, 75, 90))
print(format_grade_report("Sara", "Lee", 55, 65, 75))
print(format_grade_report("Tom", "Wong", 90, 85, 95))
print(format_grade_report("Kim", "Park", 70, 70, 70))
print(format_grade_report("Eva", "Chan", 50, 60, 70))
print(format_grade_report("Mike", "Kim", 100, 90, 80))
```

Set 10 ⚡ COMBINATION — Build an Order Summary Generator

Chapter: *Ch.3 + Ch.4 — Combination Challenge* | **Framework:** *Ch.3 Employee Profile + Ch.4 Bill Splitter*
Function: `calculate_order(product_code, unit_price, quantity, discount_rate)`

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def calculate_order(product_code, unit_price, quantity, discount_rate):
    category = product_code[0:3].upper()
    subtotal = unit_price * quantity
    discount = subtotal * (discount_rate / 100)
    total = round(subtotal - discount, 2)
    return f"[{category}] Total: {total}"

print(calculate_order("electronics001", 50, 4, 10))
print(calculate_order("books002", 20, 5, 0))
print(calculate_order("clothing003", 80, 3, 25))
print(calculate_order("food004", 10, 10, 5))
print(calculate_order("sports005", 100, 2, 20))
print(calculate_order("toys006", 30, 6, 0))
print(calculate_order("medical007", 150, 1, 15))
```

Set 11 ⚡ COMBINATION — Build an Energy Bill Calculator

Chapter: *Ch.4 + Ch.5 — Combination Challenge* | **Framework:** *Ch.4 Bill Splitter + Ch.5 Conditionals*
Function: `calculate_energy_bill(units_used, is_residential)`

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def calculate_energy_bill(units_used, is_residential):
    if not isinstance(units_used, (int, float)):
        return "Invalid units"
    if not isinstance(is_residential, bool):
        return "Invalid account type"
    if units_used < 0:
        return "Invalid units"
    if is_residential:
        rate = 0.12
    else:
        rate = 0.22
    cost = units_used * rate
    if units_used > 500:
        cost += 15.0
```

```
        return round(cost, 2)

print(calculate_energy_bill("500", True))
print(calculate_energy_bill(500, "yes"))
print(calculate_energy_bill(-100, True))
print(calculate_energy_bill(200, True))
print(calculate_energy_bill(200, False))
print(calculate_energy_bill(600, True))
print(calculate_energy_bill(600, False))
```

Set 12 ⚡ COMBINATION — Build a Final Grade Calculator

Chapter: *Ch.2 + Ch.5 + Ch.6 — Multi-Chapter Challenge* | **Framework:** *Ch.2 Report Card + Ch.5 Conditionals + Ch.6 Functions*

Function: `calculate_final_grade(raw_score, bonus_marks, has_attended_all)`

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def calculate_final_grade(raw_score, bonus_marks, has_attended_all):
    if not isinstance(raw_score, (int, float)):
        return "Invalid score"
    if not isinstance(bonus_marks, (int, float)):
        return "Invalid bonus"
    if not isinstance(has_attended_all, bool):
        return "Invalid attendance status"
    if raw_score < 0 or raw_score > 100:
        return "Score out of range"
    adjusted = raw_score + bonus_marks
    if has_attended_all:
        adjusted += 5
    if adjusted >= 90:
        grade = "A"
    elif adjusted >= 80:
        grade = "B"
    elif adjusted >= 70:
        grade = "C"
    elif adjusted >= 60:
        grade = "D"
    else:
        grade = "F"
    return f"Score: {adjusted:.1f} | Grade: {grade}"

print(calculate_final_grade("80", 5, True))
print(calculate_final_grade(80, "five", True))
print(calculate_final_grade(80, 5, "yes"))
print(calculate_final_grade(110, 5, True))
print(calculate_final_grade(85, 5, True))
print(calculate_final_grade(75, 0, False))
print(calculate_final_grade(50, 5, False))
```